Machine Learning - Milestone #2

Task #1: Proper separation of the dataset (data splitting)

What was done? In the first step, the target variable (**OutcomeType**) was separated from the input features (as it was required by the "split" method), and columns not participating in model training were removed. Subsequently, a 5-fold Stratified K-Fold cross-validation split was performed on the training dataset.
Why was it necessary? Stratified splitting guarantees that the target variable class proportions in each created subset (fold) match the distribution of the entire dataset. This prevents the model from being biased against an underrepresented class.
How was it implemented? Using the "StratifiedKFold" module from the "sklearn" library. Records received an identifier (from 0 to 4) in a new column named "fold". The success of the split was validated by checking the distribution of the target variable.

	Total %	Fold 0 %
OutcomeType		
Adoption	40.289573	40.273101
Transfer	35.250103	35.260007
Return_to_owner	17.905646	17.919940
Euthanasia	5.817651	5.817434
Died	0.737027	0.729517

Task #2: Improving data quality (data cleaning)

What was done? In this phase, handling of missing values (imputation) and standardization of inconsistent data formats took place. Missing values for **Name** and **SexuponOutcome** were replaced with placeholders, while ages recorded in text format (**AgeuponOutcome**) were converted into a calculated, continuous numerical value (**AgeInMonths**), with its missing values imputed using the median.
Why was it necessary? Since most machine learning algorithms cannot process missing data or natural language units, these foundational transformations were essential. Specifically, months were chosen as the unit of measurement because they provide a balanced granularity across the varied lifespans of shelter animals—measuring in days would create extremely large numbers for senior animals, while years would produce tiny fractional values for newborns. Additionally, the median (grouped by animal type) was selected over the mean for imputation because animal age distributions are typically right-skewed, dominated by very young animals with only a few older outliers. The median is highly resistant to these extremes, making it a much more robust metric for central tendency.
How was it implemented? Ages were converted into a continuous variable in months (**AgeInMonths**) by a custom function (parse_age_in_months). Missing values in categorical

columns (**Name**, **SexuponOutcome**) were filled with constant placeholder strings ("No Name", "Unknown"). Numerical missing values were imputed using the median according to the animal's species (**AnimalType**). Special attention was paid to avoiding data leakage: the imputation of the training set was performed fold-by-fold so that the median was always calculated from data outside the examined fold. For the imputation of the test set, the global median derived from the entire training set was applied.

Task #3: Data transformation

What was done? We executed the normalization/scaling of numerical variables, the numerical encoding of categorical variables, and created new features with higher information content from the original data.
Why was it necessary? Distance- and gradient-based algorithms are highly sensitive to magnitude differences in input variables; without scaling, features with larger numerical values would unjustifiably dominate the model. High-cardinality categorical data require special representation to avoid the "curse of dimensionality", where too many new columns would degrade the model's generalization ability.
How was it implemented? To guarantee consistent transformation, the training and test sets were combined (“df_all”) for the duration of the operations. The specific transformations for each feature were implemented as follows:

- **AgeInMonths**: To handle data skewness, a logarithmic transformation (“log1p”) was applied first. A “StandardScaler” was then used to standardize the values, resulting in the new **AgeInMonths_scaled** feature representing a normalized distribution.
- **AnimalType** and **SexuponOutcome**: As low-cardinality categorical features, these were transformed using One-Hot Encoding (OHE). This generated binary (0/1) indicator columns, and the first category was dropped to prevent redundancy.
- **Breed** and **Color**: Due to the high cardinality of these columns, Frequency Encoding was applied instead of OHE. The original categorical values were replaced with their relative frequencies within the dataset, yielding the continuous numerical features **Breed_freq** and **Color_freq**.
- **Name**: A binary logic approach was applied to extract higher-level information rather than keeping specific names. A new column named **has_name** was created, assigning a value of 1 if the animal possessed a unique name, and 0 if the original value was "No Name".

Task #4: Creation of new features

What was done? New continuous and binary features were engineered from the DateTime and Breed columns.
Why was it necessary? Raw timestamps and complex text attributes cannot be directly interpreted by machine learning models. Decomposing timestamps captures seasonal and operational patterns, while breed indicators isolate mixed-breed traits that heavily influence adoption outcomes.
How was it implemented?

- **Temporal features:** Extracted month (DateTime_month), weekday (DateTime_weekday), and hour (DateTime_hour) from DateTime using pandas datetime accessors.
- **Breed indicator:** Applied regular expression matching (\bMix\b) to the Breed column to create the binary feature

Task #5: Feature Selection

What was done? Redundant, set-specific, and intermediate columns were dropped, and the combined dataset was split back into the final training and test sets based on the target variable (OutcomeType).
Why was it necessary? Retaining unnecessary unique identifiers, raw text attributes, or target-dependent variables causes data leakage, increases model complexity, and degrades generalization performance on unseen data.
How was it implemented?

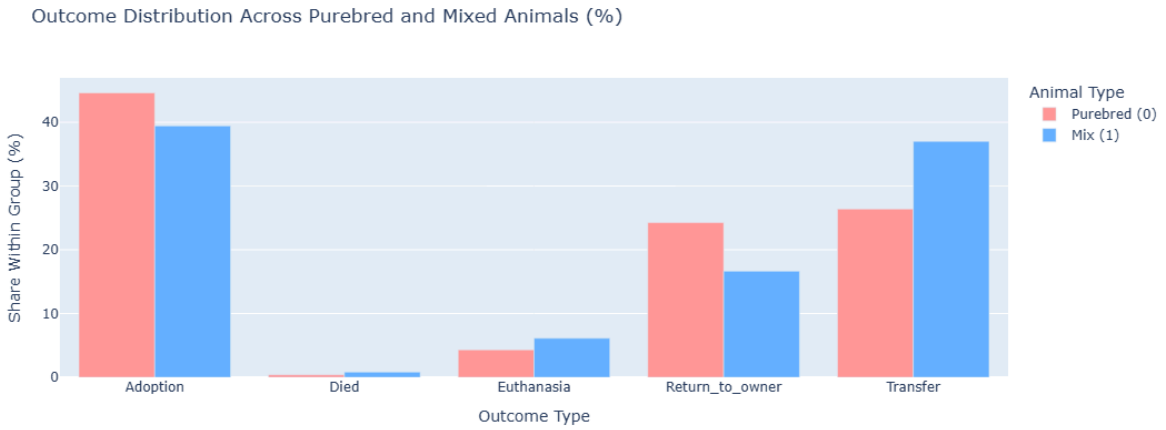
- Columns no longer needed (such as raw names, original timestamps, unscaled variables, and unique IDs) were filtered out using pandas drop operations.
- The dataset was split into train_transformed and test_transformed based on the presence of OutcomeType.
- The final processed datasets were exported to CSV format for subsequent model training.

Task #6: Feature Analysis

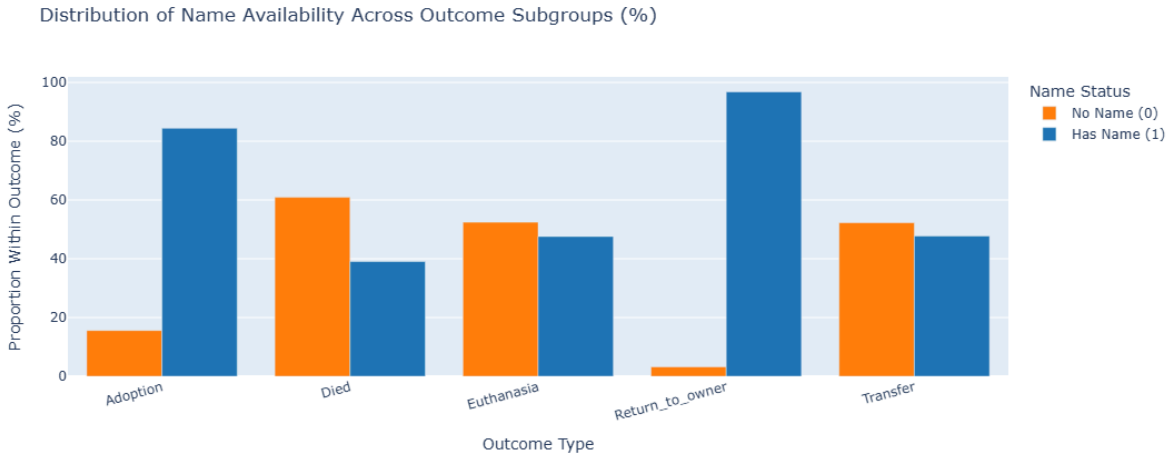
What was done? Target-driven exploratory data analyses were conducted on the engineered features **(isMix, has_name, DateTime_month, DateTime_weekday, and DateTime_hour)** to evaluate their behavioral trends and predictive distributions against the target variable **(OutcomeType)**.
Why was it necessary? Visualizing data distributions confirms whether the newly created features contain meaningful predictive signals and helps uncover hidden operational patterns within the shelter dataset.
How was it implemented?

- **Categorical and binary impact:** Grouped bar charts were used to inspect outcome distributions across mixed-breed vs. purebred groups and named vs. unnamed animals.
- **Temporal seasonality:** Line charts (by month, day of the week, and hour) were used to track shifts and shelter activity cycles over time.

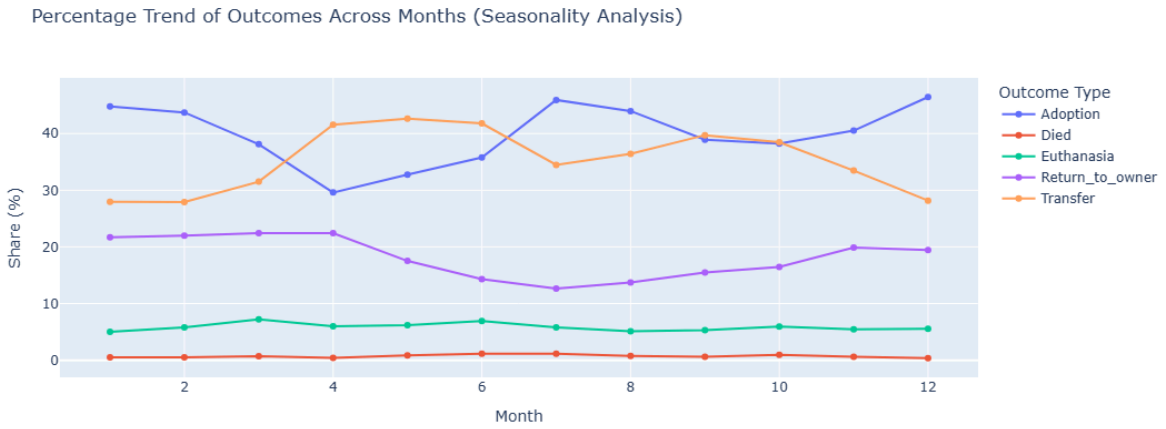
Key findings from the visualisations



Purebred animals have a slightly higher adoption rate (~45% vs. ~39%) and a noticeably higher rate of being returned to their owners (~24% vs. ~17%).
Mixed-breed animals are transferred to other facilities at a significantly higher proportion (~37% vs. ~26%).



Possessing a name is a strong indicator of positive outcomes: ~84% of adopted animals and ~97% of animals returned to their owners have assigned names.
Conversely, unnamed animals dominate negative or transitional categories like Euthanasia (~52%), Died (~61%), and Transfer (~52%).



Adoption rates peak during mid-summer and late winter/year-end. Transfer rates show an inverse trend, peaking in spring/early summer.

Task #7: Feature space

What was done? The final dataset size was checked, and a correlation matrix heatmap was created for all numerical and binary features to spot redundant variables.
Why was it necessary? Highly correlated features can confuse machine learning models, cause overfitting, and slow down training. Therefore, it is important to check for highly correlated features and remove them if necessary. Ensuring a clean feature space leads to a more stable and efficient model.
How was it implemented?

- **Dimensions check:** The total sample count and active feature columns of **train_transformed** were summarized (excluding auxiliary variables like **fold**).
- **Correlation heatmap:** Pearson correlation coefficients were calculated for all numeric features and visualized using a heatmap
- **Multicollinearity filtering:** An automated threshold search (> 0.85) was applied to the upper triangle of the correlation matrix to detect and flag potentially redundant feature pairs.